Long Chain-of-Thought Compression via Fine-Grained Group Policy Optimization

Xinchen Han [1], Hossam Afifi [1], Michel Marot [1], Xilu Wang [2], Lu Yin [2]

[1]Institut Polytechnique de Paris [2]University of Surrey

Email: isxinchen.han@gmail.com

Code: https://github.com/Mr-XcHan/FGO

Motivation and Core Idea

LLMs often generate unnecessarily verbose Chain-of-Thought (CoT) reasoning. Longer CoTs increase latency and cost, while reasoning accuracy does not necessarily scale with token length.

- **Goal:** compress long CoTs while preserving or even improving reasoning performance.
- **Method:** Fine-Grained Group Policy Optimization (FGO), an RL post-training algorithm built on GRPO.
- **Key mechanism:** regroup sampled responses into correct and incorrect subgroups, then assign length- and entropy-aware fine-grained rewards.
- **Main effect:** FGO improves token efficiency, avoids zero-advantage groups, and mitigates entropy collapse.

GRPO Limitations

GRPO defines the advantage of the i-th response as

$$A_{i,t} = \frac{r_i - \text{mean}(\{r_i\}_{i=1}^G)}{\text{std}(\{r_i\}_{i=1}^G)}.$$

GRPO then adopts the following clipped objective:

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}_{(s,a)\sim\mathcal{D},\{o_i\}_{i=1}^G\sim\pi_{\theta_{old}}(\cdot|s)}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\left(\min\left(\rho_{i,t}(\theta)A_{i,t},\ \text{clip}\left(\rho_{i,t}(\theta),1-\varepsilon,1+\varepsilon\right)A_{i,t}\right) - \gamma D_{\text{KL}}(\pi_\theta||\pi_{\text{ref}})\right)\right],$$

Inefficient data utilization. When all responses in a group receive identical rewards, all advantages become zero.

Entropy collapse. During training, the response entropy decreases sharply.

Fine-Grained Group Policy Optimization

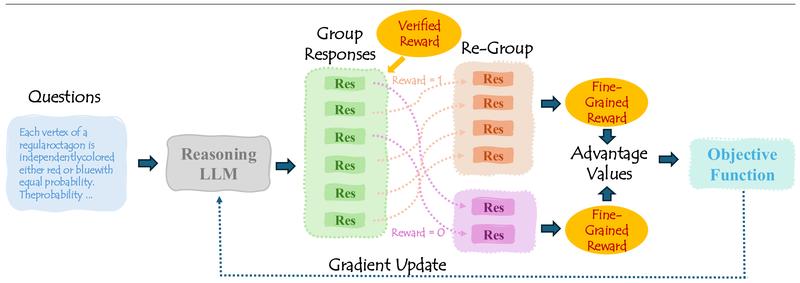

Figure 1. FGO pipeline: sample groups of responses, verify answers, regroup by correctness, assign length/entropy-aware fine-grained rewards, and optimize the GRPO-style objective.

FGO regrouping. FGO first verifies extracted answers and forms

$$\mathcal{G}^+ = \{\hat{a}_i \mid r_i = 1\}, \qquad \mathcal{G}^- = \{\hat{a}_i \mid r_i = -1\}.$$

Length- and entropy-aware fine-grained rewards. For correct responses, FGO favors shorter and more confident (lower-entropy) reasoning:

$$\mathcal{W}^+ = \text{Softmax}\left[\left(\frac{\text{mean}(L^+)}{L^+}\right)^\alpha\left(\frac{\text{mean}(\mathcal{H}^+)}{\mathcal{H}^+}\right)^\beta\right], \qquad R^+ = \mathcal{W}^+ \times \bar{R}^+.$$

For incorrect responses, shorter and more exploratory (i.e., higher-entropy) responses receive greater weight:

$$\mathcal{W}^- = \text{Softmax}\left[\left(\frac{L^-}{\text{mean}(L^-)}\right)^\alpha\left(\frac{\text{mean}(\mathcal{H}^-)}{\mathcal{H}^-}\right)^\beta\right], \qquad R^- = \mathcal{W}^- \times \bar{R}^-.$$

- α controls CoT length compression; larger α generally pushes shorter responses.
- β controls the entropy-aware weighting and helps maintain exploration.
- The final advantage follows Dr. GRPO-style normalization:

$$A_{i,t} = r_i - \text{mean}(\{r_i\}_{i=1}^G), \qquad r_i \in R^+ \cup R^-.$$

Case Study: Shorter Reasoning, Correct Answer

Question: Below is the graph of $y = a \sin (bx + c) + d$ for some positive constants $a,$ $b,$ $c,$ and $d.$ Find the smallest possible value of $c.$ [asy]import TrigMacros; size(400); real f(real x) { return 2*sin(3*x + pi) + 1; } draw(graph(f,-3*pi,3*pi,n=700,join=operator ..),red); trig_axes(-3*pi,3*pi,-4,4,pi/2,1); layer(); rm_trig_labels(-5,5, 2); label("1", (0,1), E); label("2", (0,2), E); label("3", (0,3), E); label("-1", (0,-1), E); label("-2", (0,-2), E); label("-3", (0,-3), E); [/asy]

Answer: We see that the graph reaches its midpoint at $x = 0.$ It is also decreasing at $x = 0.$ The graph of $y = \sin x$ first reaches its midpoint at $x = \pi$ for positive values of x (and is decreasing at this point), so the smallest possible value of c is $\boxed{\pi}.$

Vanilla (Token Length: 687)

Okay, so I have this graph of y = a sin(bx + c) + d, and I need to find the smallest possible value of c. The graph is drawn using the Asymptote code, which defines the function as 2*sin(3*x + pi) + 1. So, let's analyze this function.
First...
Wait, but the given function is y = 2*sin(3x + pi) + 1. Let's see what this graph looks like.
...
Wait, the phase shift is -c/b, so if we want to find c, we can set -c/b = -pi/3 + 2pi*k, where k is an integer. So, c = pi/3 + 2pi*k. Since we are looking for the smallest possible positive value of c, we can take k = 0, so c = pi/3.
...
Wait, but the Asymptote code has c = pi, so perhaps the given c is pi, but we need to find the smallest possible c.
Wait, let's think differently.
...
To find the smallest positive c, we can take k = 0, so c = pi/3. Therefore, the smallest possible value of c is pi/3.
The smallest possible value of \(c \) is \(**boxed{\frac{\pi}{3}**\). ✗

GRPO (Token Length: 613)

Okay, so I have this graph of y = a sin(bx + c) + d, and I need to find the smallest possible value of c. The graph is drawn using the Asymptote code, which defines the function as 2*sin(3*x + pi) + 1. So, let's analyze this function.
First...
Wait, but the given function is y = 2*sin(3x + pi) + 1. Let's see what this graph looks like. The amplitude is 2, so it goes from 1 - 2 = -1 to 1 + 2 = 3.
...
The period is 2pi/b = 2pi/3. The phase shift is -c/b = -pi/3, so the graph is shifted to the left by pi/3. The vertical shift is 1.
...
Wait, the period is 2pi/3, so the phase shift can be adjusted by adding or subtracting multiples of 2pi/3. So, the general form of the phase shift is -c/b = -pi/3 + 2pi*k, where k is an integer.
...
So, the smallest positive value of c is pi/3. Therefore, the smallest possible value of c is pi/3. The smallest possible value of \(c \) is \(**boxed{\frac{\pi}{3}**\). ✗

FGO (Token Length: 261)

Okay, so I have this graph of y = a sin(bx + c) + d, and I need to find the smallest possible value of c. The asy code shows that a = 2, b = 3, c = pi, and d = 1. But I need to find the smallest c.
First, I know that the general form is y = a sin(bx + c) + d. The phase shift is -c/b. So the graph is shifted left by c/b. From the Asy code, c = pi. So c/b = pi/3. So the shift is -pi/3.
But the problem asks for the smallest possible c.
So I need to find the smallest c such that the graph matches the given Asy code.
Since a, b, c, d are positive, and the graph is given, c must be such that the sine function is shifted appropriately.
So the smallest c would be when c is minimized, likely c = -pi, but since c is positive, the smallest positive c is c = pi.
Wait, no, c can be any positive constant, so the smallest c is c = pi. So the answer is c = pi. **\boxed{\pi}** ✓

Figure 2. On a MATH500 problem, FGO answers correctly with a shorter CoT, while Vanilla and GRPO produce longer but incorrect responses.

Experimental Setup

- **Models:** Qwen2.5-Math-1.5B / -Instruct, DeepSeek-R1-Distill-Qwen-1.5B, and ZR1-1.5B.
- **Training data:** the first $3,200$ samples from MATH-Lighteval.
- **Evaluation:** MATH500, AIME24, AMC23, and Minerva.
- **Metrics:** accuracy, token length, and ACT, where ACT measures accuracy contribution per 100 tokens.

Main Results

FGO improves token efficiency across four mathematical reasoning benchmarks. ACT denotes accuracy contribution per 100 tokens; higher ACT means better accuracy per generated token.

Model	Algo	Math500			AIME24			AMC23			Minerva		
		Acc	Length	ACT	Acc	Length	ACT	Acc	Length	ACT	Acc	Length	ACT
Qwen2.5-Math-1.5B	Vanilla	40.0	763	5.2	10.0	1283	0.8	30.0	978	3.1	9.2	1357	0.7
	GRPO	65.6	578	11.3	13.6	1017	1.3	50.0	825	6.1	11.4	923	1.2
	TLDR	68.2	686	9.9	-	-	-	-	-	-	-	-	-
	FGO	**68.6**	**441**	**15.6**	**19.3**	**946**	**2.0**	**52.5**	**686**	**7.7**	**18.0**	**673**	**2.7**
DeepSeek-R1-Distill-Qwen-1.5B	Vanilla	32.4	982	3.3	3.7	1889	0.2	27.5	1820	1.5	12.1	1404	0.9
	GRPO	51.0	828	6.2	7.7	1879	0.4	30.0	1646	1.8	**20.6**	1132	1.8
	TLDR	54.6	308	17.7	0.0	**494**	0	19.3	**444**	4.3	-	-	-
	FGO	**56.4**	**229**	**24.6**	**8.3**	962	**0.9**	**40.0**	651	**6.1**	18.0	**202**	**8.9**
ZR1-1.5B	Vanilla	58.4	704	8.3	13.0	1840	0.7	50.0	1450	3.4	17.3	1235	1.4
	GRPO	64.0	706	9.1	13.3	1840	0.7	50.0	1453	3.4	23.2	1209	1.9
	TLDR	-	-	-	-	-	-	-	-	-	-	-	-
	FGO	**69.0**	**321**	**21.5**	**13.7**	**1151**	**1.2**	**55.0**	**540**	**10.2**	**24.6**	**412**	**6.0**
Qwen2.5-Math-1.5B-Instruct	Vanilla	71.2	542	13.1	12.7	890	1.4	47.5	802	5.9	20.2	652	3.1
	GRPO	73.0	541	13.5	**14.0**	998	1.4	52.5	746	7.0	22.1	645	3.4
	TLDR	69.7	603	11.6	7.0	1031	0.7	37.3	861	4.3	-	-	-
	FGO	**73.2**	**321**	**22.8**	13.3	**810**	**1.6**	**55.0**	**494**	**11.1**	**23.2**	**374**	**6.2**

Table 1. Comparative results for Vanilla, GRPO, TLDR, and FGO, where bold indicates the best performance.

- FGO substantially reduces token length while maintaining, and often improving, reasoning performance.
- FGO consistently achieves the highest accuracy contribution per 100 tokens (ACT).

Eliminating Two GRPO Limitations

Data utilization. The following table shows that GRPO suffers from inefficient data utilization, whereas FGO consistently achieves 100% utilization across the $3,200$ training samples.

	Qwen2.5-Math-1.5B	DeepSeek-R1-Distill-Qwen-1.5B	ZR1-1.5B	Qwen2.5-Math-1.5B-Instruct
GRPO	1584	1866	2617	2444
FGO	0	0	0	0

Table 2. Number of invalid samples for GRPO and FGO.

Entropy preservation.

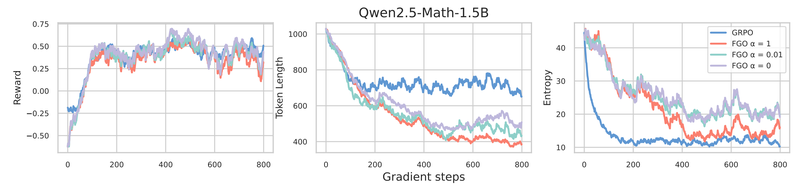

Figure 3. Training curves on Qwen2.5-Math-1.5B. FGO maintains reward, compresses length, and avoids rapid entropy collapse.

Self-Reflection Results

Self-reflection is a crucial capability in long-CoT reasoning. We therefore evaluate whether models trained with FGO preserve this behavior by counting reflection-related keywords such as *"wait"*, *"alternatively"*, *"emm"*, and *"hmm"* per 100 tokens. FGO preserves most self-reflection steps and maintains reasoning capability while compressing CoT length.

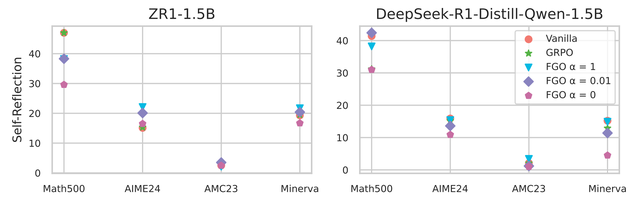

Figure 4. FGO keeps most self-reflection signals while compressing CoT length.

Conclusion

- **FGO compresses CoT reasoning** by assigning fine-grained rewards to regrouped correct and incorrect responses.
- **FGO improves GRPO training** by addressing inefficient data utilization and entropy collapse.
- **FGO preserves reasoning quality** while substantially reducing response length across multiple benchmarks.
- **Future work** will explore more accurate advantage estimation with fewer sampled responses per group.

Let's connect

PhD finishing soon; open to research roles in RL and LLM reasoning.

isxinchen.han@gmail.com

Interests: Offline RL ▪ LLM post-training ▪ Agentic AI